

All You Need Is Superword-Level Parallelism: Systematic Control-Flow Vectorization with SLP (PLDI '22)

Yishen Chen¹ Charith Mendis² Saman Amarasinghe¹

¹Massachusetts Institute of Technology

²University of Illinois at Urbana-Champaign

January 16, 2025

Outline

Introduction and Motivation

- Current State of Autovectorization
- Flexibility of SLP Vectorization
- Key Contributions

Groundwork for SuperVectorization

- Handling Control Flow in SLP Vectorization
- Introduction to Predicated SSA

SuperVectorization

- Conversion to Predicated SSA
- Performing Vectorization
- Converting Back to CFG
- Performance Evaluation

Takeaways

Outline

Introduction and Motivation

- Current State of Autovectorization
- Flexibility of SLP Vectorization
- Key Contributions

Groundwork for SuperVectorization

- Handling Control Flow in SLP Vectorization
- Introduction to Predicated SSA

SuperVectorization

- Conversion to Predicated SSA
- Performing Vectorization
- Converting Back to CFG
- Performance Evaluation

Takeaways

Current State of Autovectorization

A tale of two methodologies

- Two methods of autovectorization: **loop vectorization** and **SLP vectorization**

Current State of Autovectorization

A tale of two methodologies

- Two methods of autovectorization: **loop vectorization** and **SLP vectorization**
- Loop vectorization: targets **loop-level parallelism**
 - Influenced by long-vector architectures (e.g., CRAY)
 - Works by widening scalar instructions in loop body to vectorized counterparts
 - Requires complex dependence analyses

Current State of Autovectorization

A tale of two methodologies

- Two methods of autovectorization: **loop vectorization** and **SLP vectorization**
- Loop vectorization: targets **loop-level parallelism**
 - Influenced by long-vector architectures (e.g., CRAY)
 - Works by widening scalar instructions in loop body to vectorized counterparts
 - Requires complex dependence analyses
- Superword-level parallelism vectorization: exploits **short-vector parallelism**
 - Influenced by modern vector extensions (e.g., Intel AVX)
 - Works by packing independent and isomorphic groups of instructions
 - More flexible, but harder to handle control flow

Current State of Autovectorization

A tale of two methodologies

- Two methods of autovectorization: **loop vectorization** and **SLP vectorization**
- Loop vectorization: targets **loop-level parallelism**
 - Influenced by long-vector architectures (e.g., CRAY)
 - Works by widening scalar instructions in loop body to vectorized counterparts
 - Requires complex dependence analyses
- Superword-level parallelism vectorization: exploits **short-vector parallelism**
 - Influenced by modern vector extensions (e.g., Intel AVX)
 - Works by packing independent and isomorphic groups of instructions
 - More flexible, but harder to handle control flow
- The two techniques have their advantages and disadvantages
 - Production compilers use both techniques in separate passes

Current State of Autovectorization

Loop vectorization in action

- SAXPY loop widened by factor of 8

```
void saxpy(float a, float x[], float y[],
           int n) {
    for (int i = 0; i < n; i++)
        y[i] = a * x[i] + y[i];
}
```


Current State of Autovectorization

Loop vectorization in action

- SAXPY loop widened by factor of 8

```
void saxpy(float a, float x[], float y[],
           int n) {
    for (int i = 0; i < n; i++)
        y[i] = a * x[i] + y[i];
}
```

```
void saxpy(float a, float x[], float y[],
           int n) {
    int i;
    for (i = 0; i + 8 <= n; i += 8)
        y[i:i + 8] = splat(a, 8) * x[i:i + 8] +
            y[i:i + 8];
    for (; i < n; i++)
        y[i] = a * x[i] + y[i];
}
```

Current State of Autovectorization

SLP vectorization in action

- Four elements grouped and processed together

```
void foo(int a1, int a2, int b1, int b2,
        int *A) {
    A[0] = a1 * (a1 + b1);
    A[1] = a2 * (a2 + b2);
    A[2] = a1 * (a1 + b1);
    A[3] = a2 * (a2 + b2);
}
```

Current State of Autovectorization

SLP vectorization in action

- Four elements grouped and processed together

```
void foo(int a1, int a2, int b1, int b2,  
        int *A) {  
    A[0] = a1 * (a1 + b1);  
    A[1] = a2 * (a2 + b2);  
    A[2] = a1 * (a1 + b1);  
    A[3] = a2 * (a2 + b2);  
}
```

```
void foo(int a1, int a2, int b1, int b2,  
        int *A) {  
    va = {a1, a2, a1, a2};  
    vb = {b1, b2, b1, b2};  
    A[0:4] = va * (va + vb);  
}
```

Flexibility of SLP Vectorization

Inefficiency of loop vectorization

- Naive vectorization of unrolled SAXPY code

```
void saxpy(float a, float x[], float y[],
           int n) {
    for (int i = 0; i + 2 <= n; i += 2) {
        y[i] = a * x[i] + y[i];
        y[i + 1] = a * x[i + 1] + y[i + 1];
    }
    // handle leftover elements
}
```

Flexibility of SLP Vectorization

Inefficiency of loop vectorization

- Naive vectorization of unrolled SAXPY code

```
void saxpy(float a, float x[], float y[],
           int n) {
    for (int i = 0; i + 2 <= n; i += 2) {
        y[i] = a * x[i] + y[i];
        y[i + 1] = a * x[i + 1] + y[i + 1];
    }
    // handle leftover elements
}
```

```
void saxpy(float a, float x[], float y[],
           int n) {
    int i;
    for (i = 0; i + 16 <= n; i += 16) {
        vx = x[i:i + 16]; vy = y[i:i + 16];
        vx_even = shuffle_even(vx);
        vy_even = shuffle_even(vy);
        vx_odd = shuffle_odd(vx);
        vy_odd = shuffle_odd(vy);
        y[i:i + 16] = interleave(
            splat(a, 8) * vx_even + vy_even,
            splat(a, 8) * vx_odd + vy_odd
        );
    }
    // handle leftover elements
}
```

Flexibility of SLP Vectorization

Flexibility of unroll-and-SLP

- After unrolling 4 times, SLP vectorization leads to better code

```
void saxpy(float a, float x[], float y[],
           int n) {
    for (int i = 0; i < n; i += 8) {
        y[i] = a * x[i] + y[i];
        y[i + 1] = a * x[i + 1] + y[i + 1];
        y[i + 2] = a * x[i + 2] + y[i + 2];
        y[i + 3] = a * x[i + 3] + y[i + 3];
        y[i + 4] = a * x[i + 4] + y[i + 4];
        y[i + 5] = a * x[i + 5] + y[i + 5];
        y[i + 6] = a * x[i + 6] + y[i + 6];
        y[i + 7] = a * x[i + 7] + y[i + 7];
    }
    // handle leftover elements
}
```

Flexibility of SLP Vectorization

Flexibility of unroll-and-SLP

- After unrolling 4 times, SLP vectorization leads to better code

```
void saxpy(float a, float x[], float y[],
           int n) {
    for (int i = 0; i < n; i += 8) {
        y[i] = a * x[i] + y[i];
        y[i + 1] = a * x[i + 1] + y[i + 1];
        y[i + 2] = a * x[i + 2] + y[i + 2];
        y[i + 3] = a * x[i + 3] + y[i + 3];
        y[i + 4] = a * x[i + 4] + y[i + 4];
        y[i + 5] = a * x[i + 5] + y[i + 5];
        y[i + 6] = a * x[i + 6] + y[i + 6];
        y[i + 7] = a * x[i + 7] + y[i + 7];
    }
    // handle leftover elements
}
```

```
void saxpy(float a, float x[], float y[],
           int n) {
    int i;
    for (i = 0; i + 8 <= n; i += 8)
        y[i:i + 8] = splat(a, 8) * x[i:i + 8] +
            y[i:i + 8];
    // handle leftover elements
}
```

TL;DR

Key contributions

1. **Predicated SSA** for simplifying code motion across arbitrary control flow regions
2. **SuperVectorization**, a generalized form of SLP vectorization capable of handling inner and outer loops as well as straight-line code
3. Prototype implementation which outperforms LLVM's vectorization passes

Outline

Introduction and Motivation

- Current State of Autovectorization
- Flexibility of SLP Vectorization
- Key Contributions

Groundwork for SuperVectorization

- Handling Control Flow in SLP Vectorization
- Introduction to Predicated SSA

SuperVectorization

- Conversion to Predicated SSA
- Performing Vectorization
- Converting Back to CFG
- Performance Evaluation

Takeaways

Shortcomings of Conventional SLP Vectorization

- Conventional SLP vectorization has been limited to straight-line code
 - However, SLP is not just about similar instructions in a single basic block!

Shortcomings of Conventional SLP Vectorization

- Conventional SLP vectorization has been limited to straight-line code
 - However, SLP is not just about similar instructions in a single basic block!

```
void search(int haystack[], int needles[2], int needle_idx[2], int n) {  
    for (int i = 0; i < n; i++)  
        if (haystack[i] == needles[0]) {  
            needle_idx[0] = i;  
            break;  
        }  
    for (int i = 0; i < n; i++)  
        if (haystack[i] == needles[1]) {  
            needle_idx[1] = i;  
            break;  
        }  
}
```

- The two loops are independent and, thus can be run in parallel

Handling Control Flow in SLP Vectorization

Key challenges

- **Code motion** across basic blocks is challenging
 - Packing instructions involve code motion
 - Exploiting SLP across basic block boundaries means code motion across BBs
 - In traditional IR with CFG this requires restructuring CFG, which is complex
- **If-conversion**: partially convert control flow into straight-line code
 - ϕ -nodes are converted into select instructions
 - Analogous to converting `if (cond) f(); else g();` into `cond ? f() : g();`
 - All branches have to be executed regardless of condition

Handling Control Flow in SLP Vectorization

Key challenges

- **Code motion** across basic blocks is challenging
 - Packing instructions involve code motion
 - Exploiting SLP across basic block boundaries means code motion across BBs
 - In traditional IR with CFG this requires restructuring CFG, which is complex
- **If-conversion**: partially convert control flow into straight-line code
 - ϕ -nodes are converted into select instructions
 - Analogous to converting `if (cond) f(); else g();` into `cond ? f() : g();`
 - All branches have to be executed regardless of condition
- Solution: **Predicated SSA** for code motion and conditional if-conversion

Introduction to Predicated SSA

Definition

- Basic idea: “flatten” the CFG to expose SLP, while retaining control dependences

Introduction to Predicated SSA

Definition

- Basic idea: “flatten” the CFG to expose SLP, while retaining control dependences
- Remove distinctions of basic blocks by introducing **control predicates**
 - Annotates each instruction as boolean flags determining whether to execute

Introduction to Predicated SSA

Definition

- Basic idea: “flatten” the CFG to expose SLP, while retaining control dependences
- Remove distinctions of basic blocks by introducing **control predicates**
 - Annotates each instruction as boolean flags determining whether to execute
- **Differentiate ϕ -nodes** to capture its precise semantics
 - Two main uses of ϕ -nodes: loop headers and control-flow joins
 - μ -nodes for uses in loop header, gated ϕ -nodes for control-flow joins

Introduction to Predicated SSA

Definition

$\text{fn} ::= \text{item}_1 : p_1, \dots, \text{item}_n : p_n$

$c ::= \text{branch conditions}$

$p ::= \mathbf{true} \mid c \mid \bar{c} \mid p_1 \wedge p_2 \mid p_1 \vee p_2$

$\text{item} ::= \text{instruction} \mid \text{loop}$

$\text{loop} ::= \mathbf{with} \ v_1 = \mu_1, \dots, v_m = \mu_m \ \mathbf{do} \ \text{item}_1 : p_1, \dots, \text{item}_n : p_n \ \mathbf{while} \ p_{\text{cont}}$

$\mu ::= \mathbf{mu}(v_{\text{init}}, v_{\text{rec}})$

$\phi ::= \mathbf{phi}(v_1 : p_1, \dots, v_n : p_n)$

$v ::= \mu \mid \text{instruction} \mid \text{constant} \mid \text{argument}$

Introduction to Predicated SSA

Example

```
for (int i = 0; i < n; i++)  
  if (haystack[i] == needles[0]) {  
    needle_idx[0] = i;  
    break;  
  }
```

```
with i = mu(0, i') do  
  t = load haystack[i]      : true  
  needle = load needles[0]  : true  
  found = cmp eq, t, needle : true  
  i' = add i, 1             : true  
  not_found = not found     : true  
  lt_n = cmp lt i', n       : true  
while not_found and lt_n    : true  
store i, needle_idx[0]      : found
```

Outline

Introduction and Motivation

- Current State of Autovectorization
- Flexibility of SLP Vectorization
- Key Contributions

Groundwork for SuperVectorization

- Handling Control Flow in SLP Vectorization
- Introduction to Predicated SSA

SuperVectorization

- Conversion to Predicated SSA
- Performing Vectorization
- Converting Back to CFG
- Performance Evaluation

Takeaways

Vectorization Workflow

SuperVectorization at a glance

1. Converting CFG to Predicated SSA
2. Unroll and Packing
3. Loop transformations
4. Vector instruction generation
5. Converting Predicated SSA to CFG

Conversion to Predicated SSA

High-level overview

1. Convert input IR into a canonical form resembling `do-while` loops
 - Single incoming edge and back edge, dedicated loop header, preheader, and latch

Conversion to Predicated SSA

High-level overview

1. Convert input IR into a canonical form resembling `do-while` loops
 - Single incoming edge and back edge, dedicated loop header, preheader, and latch
2. Classify ϕ -nodes according to its uses
 - Forward ϕ -nodes: use gated ϕ -nodes
 - Loop header ϕ -nodes: use μ -nodes

Conversion to Predicated SSA

High-level overview

1. Convert input IR into a canonical form resembling do-while loops
 - Single incoming edge and back edge, dedicated loop header, preheader, and latch
2. Classify ϕ -nodes according to its uses
 - Forward ϕ -nodes: use gated ϕ -nodes
 - Loop header ϕ -nodes: use μ -nodes
3. Assign and calculate control predicates for all blocks
 - Disjunction of branch conditions of control-dependent blocks' post-dominant successors
 - All instructions in a block share one control predicate

Conversion to Predicated SSA

Details of control predicate calculation

- For a basic block b to execute, two conditions must be satisfied:
 1. Control flow reaches some block b' , a control-dependent basic block of b
 2. b' takes a branch that leads to b

Conversion to Predicated SSA

Details of control predicate calculation

- For a basic block b to execute, two conditions must be satisfied:
 1. Control flow reaches some block b' , a control-dependent basic block of b
 2. b' takes a branch that leads to b
- Condition 1: check using **post-dominance frontier**

Conversion to Predicated SSA

Details of control predicate calculation

- For a basic block b to execute, two conditions must be satisfied:
 1. Control flow reaches some block b' , a control-dependent basic block of b
 2. b' takes a branch that leads to b
- Condition 1: check using **post-dominance frontier**
- Condition 2: simple to consider when all basic blocks have at most two successors
 - i.e., no switch and indirect branches

Conversion to Predicated SSA

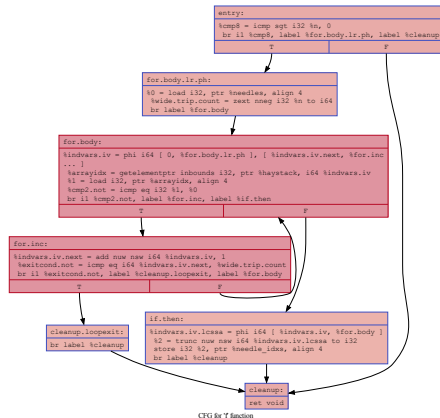
Details of control predicate calculation

- For a basic block b to execute, two conditions must be satisfied:
 1. Control flow reaches some block b' , a control-dependent basic block of b
 2. b' takes a branch that leads to b
- Condition 1: check using **post-dominance frontier**
- Condition 2: simple to consider when all basic blocks have at most two successors
 - i.e., no switch and indirect branches
 - Recall that b' is in the post-dominance frontier of b
 - There exists an edge $b' \rightarrow b''$ where b'' is a successor of b' and post-dominant on b
 - Such b'' is unique as b' has at most two outgoing edges

Conversion to Predicated SSA

Example

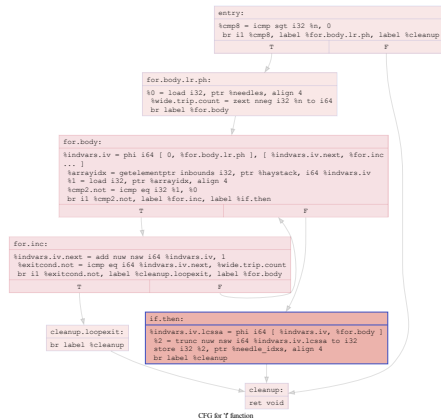
```
void search(int haystack[], int needles[2],  
           int needle_idx[2], int n) {  
    for (int j = 0; j < n; j++)  
        if (haystack[j] != needles[0]) {  
            needle_idx[0] = j;  
            break;  
        }  
}
```



Conversion to Predicated SSA

Example

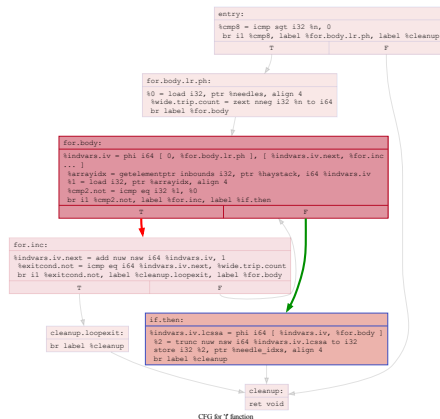
```
void search(int haystack[], int needles[2],  
           int needle_idx[2], int n) {  
    for (int j = 0; j < n; j++)  
        if (haystack[j] != needles[0]) {  
            needle_idx[0] = j;  
            break;  
        }  
}
```



Conversion to Predicated SSA

Example

```
void search(int haystack[], int needles[2],
           int needle_idx[2], int n) {
    for (int j = 0; j < n; j++)
        if (haystack[j] != needles[0]) {
            needle_idx[0] = j;
            break;
        }
}
```



Vector Packing

Overview

- Capture SLP revealed by conversion to Predicated SSA and unrolling
- SuperVectorization is compatible with arbitrary packing heuristics

Vector Packing

Overview

- Capture SLP revealed by conversion to Predicated SSA and unrolling
- SuperVectorization is compatible with arbitrary packing heuristics
 - The authors adapt the bottom-up SLP heuristic, which is simple and commonly used
 - Starting from *seeds* (i.e., contiguous stores, reductions), traverse use-def chains
 - Stop when nonvectorizable instructions are found, check dependence cycles and cost

Vector Packing

Overview

- Capture SLP revealed by conversion to Predicated SSA and unrolling
- SuperVectorization is compatible with arbitrary packing heuristics
 - The authors adapt the bottom-up SLP heuristic, which is simple and commonly used
 - Starting from *seeds* (i.e., contiguous stores, reductions), traverse use-def chains
 - Stop when nonvectorizable instructions are found, check dependence cycles and cost
- The packs are lowered into instructions using **loop transforms**, **scheduling**, and **instruction selection**

Vector Packing

Example

```
void search(int haystack[], int needles[2],
            int needle_idx[2], int n) {
    for (int j = 0; j < n; j++)
        if (haystack[j] != needles[0]) {
            needle_idx[0] = j;
            break;
        }
    for (int j = 0; j < n; j++)
        if (haystack[j] != needles[1]) {
            needle_idx[1] = j;
            break;
        }
}
```

Vector Packing

Example

```
void search(int haystack[], int needles[2],
            int needle_idxs[2], int n) {
    for (int j = 0; j < n; j++)
        if (haystack[j] != needles[0]) {
            needle_idxs[0] = j;
            break;
        }
    for (int j = 0; j < n; j++)
        if (haystack[j] != needles[1]) {
            needle_idxs[1] = j;
            break;
        }
}
```

```
with i = mu(0, i') do
    t = load haystack[i]      : true
    needle = load needles[0]  : true
    found = cmp eq, t, needle : true
    i' = add i, 1             : true
    not_found = not found     : true
    lt_n = cmp lt i', n       : true
while not_found and lt_n     : true
store i, needle_idxs[0]      : found
with i2 = mu(0, i2') do
    t2 = load haystack[i2]   : true
    needle2 = load needles[1] : true
    found2 = cmp eq, t2, needle2 : true
    i2' = add i2, 1          : true
    not_found2 = not found2  : true
    lt_n2 = cmp lt i2', n    : true
while not_found2 and lt_n2   : true
store i2, needle_idxs[1]     : found2
```

Vector Packing

Example

```
void search(int haystack[], int needles[2],
            int needle_idxs[2], int n) {
    for (int j = 0; j < n; j++)
        if (haystack[j] != needles[0]) {
            needle_idxs[0] = j;
            break;
        }
    for (int j = 0; j < n; j++)
        if (haystack[j] != needles[1]) {
            needle_idxs[1] = j;
            break;
        }
}
```

```
with i = mu(0, i') do
    t = load haystack[i]      : true
    needle = load needles[0]  : true
    found = cmp eq, t, needle : true
    i' = add i, 1             : true
    not_found = not found     : true
    lt_n = cmp lt i', n       : true
while not_found and lt_n     : true
store i, needle_idxs[0]      : found
with i2 = mu(0, i2') do
    t2 = load haystack[i2]   : true
    needle2 = load needles[1] : true
    found2 = cmp eq, t2, needle2 : true
    i2' = add i2, 1          : true
    not_found2 = not found2   : true
    lt_n2 = cmp lt i2', n     : true
while not_found2 and lt_n2   : true
store i2, needle_idxs[1]     : found2
```

Vector Packing

Example

```
void search(int haystack[], int needles[2],
            int needle_idxs[2], int n) {
    for (int j = 0; j < n; j++)
        if (haystack[j] != needles[0]) {
            needle_idxs[0] = j;
            break;
        }
    for (int j = 0; j < n; j++)
        if (haystack[j] != needles[1]) {
            needle_idxs[1] = j;
            break;
        }
}
```

```
with i = mu(0, i') do
    t = load haystack[i]      : true
    needle = load needles[0]  : true
    found = cmp eq, t, needle : true
    i' = add i, 1             : true
    not_found = not found     : true
    lt_n = cmp lt i', n       : true
while not_found and lt_n     : true
store i, needle_idxs[0]      : found
with i2 = mu(0, i2') do
    t2 = load haystack[i2]   : true
    needle2 = load needles[1] : true
    found2 = cmp eq, t2, needle2 : true
    i2' = add i2, 1          : true
    not_found2 = not found2   : true
    lt_n2 = cmp lt i2', n     : true
while not_found2 and lt_n2   : true
store i2, needle_idxs[1]     : found2
```

Vector Packing

Example

```
void search(int haystack[], int needles[2],
            int needle_idxs[2], int n) {
    for (int j = 0; j < n; j++)
        if (haystack[j] != needles[0]) {
            needle_idxs[0] = j;
            break;
        }
    for (int j = 0; j < n; j++)
        if (haystack[j] != needles[1]) {
            needle_idxs[1] = j;
            break;
        }
}
```

```
with i = mu(0, i') do
    t = load haystack[i]      : true
    needle = load needles[0]  : true
    found = cmp eq, t, needle : true
    i' = add i, 1             : true
    not_found = not found     : true
    lt_n = cmp lt i', n       : true
while not_found and lt_n     : true
store i, needle_idxs[0]      : found
with i2 = mu(0, i2') do
    t2 = load haystack[i2]   : true
    needle2 = load needles[1] : true
    found2 = cmp eq, t2, needle2 : true
    i2' = add i2, 1          : true
    not_found2 = not found2   : true
    lt_n2 = cmp lt i2', n     : true
while not_found2 and lt_n2   : true
store i2, needle_idxs[1]     : found2
```

Vector Packing

Example

```
void search(int haystack[], int needles[2],
            int needle_idxs[2], int n) {
    for (int j = 0; j < n; j++)
        if (haystack[j] != needles[0]) {
            needle_idxs[0] = j;
            break;
        }
    for (int j = 0; j < n; j++)
        if (haystack[j] != needles[1]) {
            needle_idxs[1] = j;
            break;
        }
}
```

```
with i = mu(0, i') do
    t = load haystack[i]      : true
    needle = load needles[0]  : true
    found = cmp eq, t, needle : true
    i' = add i, 1             : true
    not_found = not found     : true
    lt_n = cmp lt i', n       : true
while not_found and lt_n     : true
store i, needle_idxs[0]      : found
with i2 = mu(0, i2') do
    t2 = load haystack[i2]   : true
    needle2 = load needles[1] : true
    found2 = cmp eq, t2, needle2 : true
    i2' = add i2, 1           : true
    not_found2 = not found2   : true
    lt_n2 = cmp lt i2', n     : true
while not_found2 and lt_n2   : true
store i2, needle_idxs[1]     : found2
```

Lowering Packs into Instructions

Loop transforms: fusion and co-iteration

- Predicate SSA allows packing from different loops
 - However, the resulting vector instructions must be in the same loop

Lowering Packs into Instructions

Loop transforms: fusion and co-iteration

- Predicate SSA allows packing from different loops
 - However, the resulting vector instructions must be in the same loop
- Two ways of loop transformation: **fusion** and **co-iteration**
 - Apply fusion if loops are independent, have the same trip-count/condition, and parent loops can be fused
 - Co-iterate loops if they do not meet the conditions of fusion

Lowering Packs into Instructions

Loop transforms: fusion and co-iteration

- Predicate SSA allows packing from different loops
 - However, the resulting vector instructions must be in the same loop
- Two ways of loop transformation: **fusion** and **co-iteration**
 - Apply fusion if loops are independent, have the same trip-count/condition, and parent loops can be fused
 - Co-iterate loops if they do not meet the conditions of fusion
- Co-iteration: run the loop until all conditions of the original loops become false
 - Guard the loop bodies' execution using strengthened control predicates

Lowering Packs into Instructions

Scheduling and instruction selection

- **Scheduling:** reorder instructions to satisfy dependences introduced by packing
 - Create a dependency graph of instructions and loops
 - Since cyclic dependences have been checked before packing, the graph is DAG
 - Topological sort on the dependency graph to obtain dependency-preserving order

Lowering Packs into Instructions

Scheduling and instruction selection

- **Scheduling:** reorder instructions to satisfy dependences introduced by packing
 - Create a dependency graph of instructions and loops
 - Since cyclic dependences have been checked before packing, the graph is DAG
 - Topological sort on the dependency graph to obtain dependency-preserving order
- Special handling for load/store and ϕ -node packs in **instruction selection**
 - For other instructions, naive packing into vector registers is fine
 - Use masked loads and stores where masks are set according to control predicates
 - When lowering ϕ -nodes, emit `select` instructions if necessary

Converting Back to CFG

- After lowering the packs, the work on Predicated SSA is done

Converting Back to CFG

- After lowering the packs, the work on Predicated SSA is done
- Features specific to Predicated SSA are lowered into CFG constructs
 1. Replace gated ϕ -nodes with move instructions, ignoring SSA invariant for now
 2. Reconstruct basic blocks by querying **block builder** with control predicates
 3. Generate back edges and loop exits

Converting Back to CFG

- After lowering the packs, the work on Predicated SSA is done
- Features specific to Predicated SSA are lowered into CFG constructs
 1. Replace gated ϕ -nodes with move instructions, ignoring SSA invariant for now
 2. Reconstruct basic blocks by querying **block builder** with control predicates
 3. Generate back edges and loop exits
- Block builder: a data structure for mapping control predicates into basic blocks
 - When presented with control predicate p , the following cases are possible:

p	Block builder behavior
true	return top-level block
$p' \wedge c$	add branch with condition c to p' block
$p_1 \vee \dots \vee p_n$	join control-flow from $p_1, \dots, p_n$ blocks

Performance Evaluation

- SuperVectorization outperformed LLVM's default -O3 pipeline
 - *Manually vectorized some functions

Benchmark	TSVC	PolyBench	ISPC*
Geomean speedup	1.04×	1.46×	1.88×

Performance Evaluation

- SuperVectorization outperformed LLVM's default -O3 pipeline
 - *Manually vectorized some functions

Benchmark	TSVC	PolyBench	ISPC*
Geomean speedup	1.04×	1.46×	1.88×

- In some cases, performance regressed significantly ($< 0.2\times$)
 - Missed vectorization opportunity due to data dependences
 - Sub-optimal instruction selection (e.g., scalarized store instead of scatter)
 - Incomplete handling of some reduction patterns

Outline

Introduction and Motivation

- Current State of Autovectorization
- Flexibility of SLP Vectorization
- Key Contributions

Groundwork for SuperVectorization

- Handling Control Flow in SLP Vectorization
- Introduction to Predicated SSA

SuperVectorization

- Conversion to Predicated SSA
- Performing Vectorization
- Converting Back to CFG
- Performance Evaluation

Takeaways

Takeaways

- Dedicated representation facilitating code motion
 - Thinking outside of CFG and basic blocks might reveal new opportunities

Takeaways

- Dedicated representation facilitating code motion
 - Thinking outside of CFG and basic blocks might reveal new opportunities
- Flexibility of SLP
 - Exploiting SLP spanning multiple control flow regions had led to speedups

Takeaways

- Dedicated representation facilitating code motion
 - Thinking outside of CFG and basic blocks might reveal new opportunities
- Flexibility of SLP
 - Exploiting SLP spanning multiple control flow regions had led to speedups
- Future directions
 - Reduce dependence on loop unrolling for scalable vectorization support